Estimation via Maximization

Switching Gears to Estimation

Now that we understand the basics of coding in R, we're going to talk about estimating unknown parameters.

This material is based on the free textbook: Core Statistics

There are many approaches to parameter estimation, but we will focus on two:

- Estimation via **maximization**
- Estimation via **simulation**

Estimation via Maximization

Examples of estimation via maximization include:

- Maximum likelihood estimation
- Method of moments estimation
- Loss function based estimators
- Maximum a posteriori (MAP) estimation

Maximization vs. Minimization vs. Solving a System

We're going to treat all three as equivalent.

Maximizing something is the same as minimizing -1 times something.

Solving a system of equations is the same as minimizing absolute or squared deviations of the left hand sides and the right hand sides.

Working Through an Example

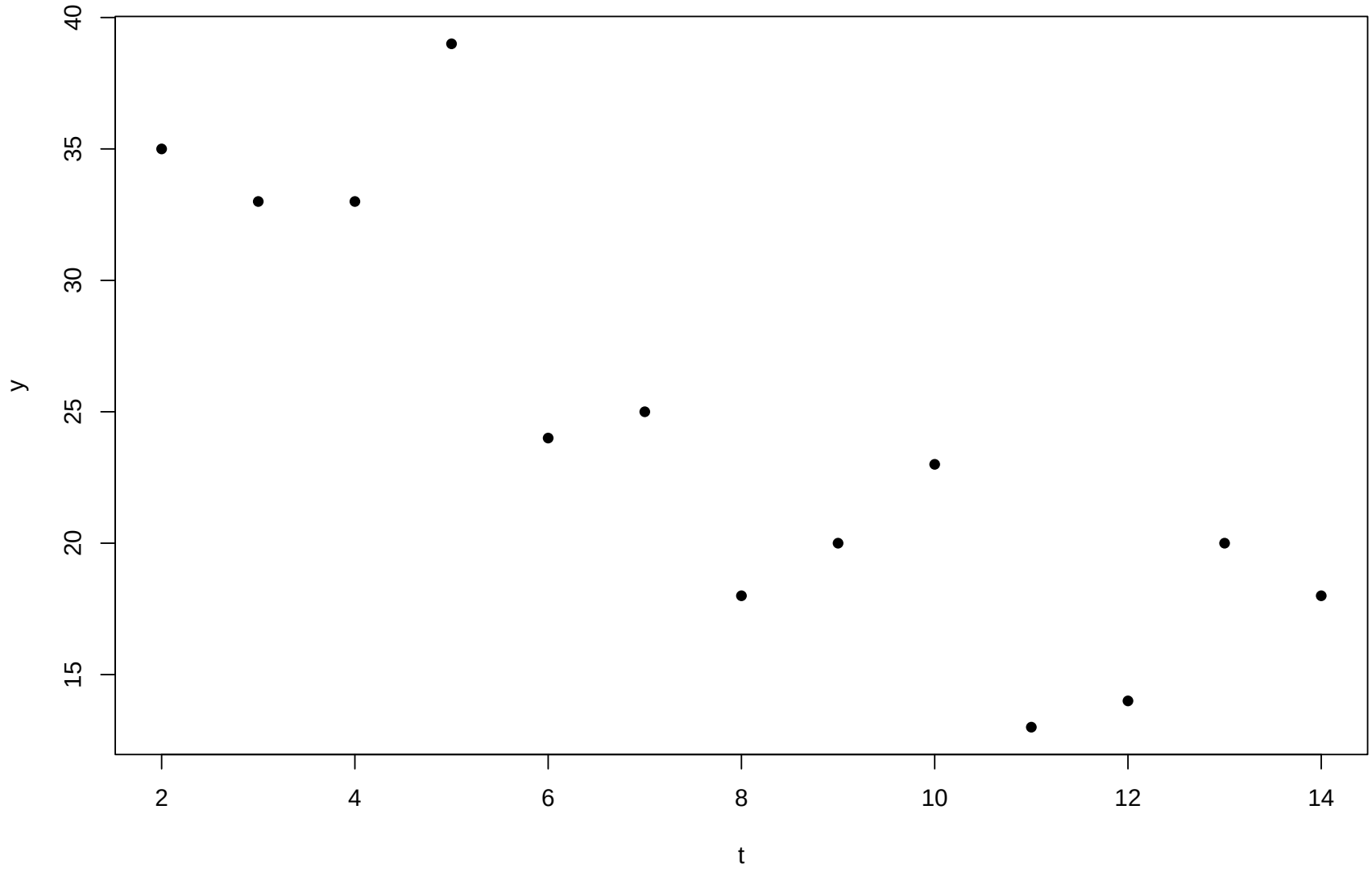
Consider some data:

```
t <- seq(2, 14, by = 1)
y <- c(35, 33, 33, 39, 24, 25,
       18, 20, 23, 13, 14, 20, 18)
```

- `y[i]` represents the i-th count of cells in a culture
- `t[i]` describes hours since treatment of the culture with an antibiotic for the i-th count

Always Visualize First

```
plot(t, y, pch = 16)
```



What does maximum likelihood require?

A statistical model for the data, in this case `y` and `t`.

The researchers want to fit the a model that describes the expected culture count as

$$E[y_i] = 50 \times \exp\{-\theta t_i\}$$

Where 50 describes the count of cells in the culture before the antibiotic was administered and the parameter is θ.

Is this a model?

A Model for Our Data

We need to assume a distribution for each y_i and we need to assume a dependence structure relating distinct elements of y_i and to each other

We assume

$$y_i \overset{indep}{\sim} \text{Poisson}(50 \times \exp\{-\theta t_i\})$$

Our First Likelihood

To start, let's pick a candidate value of θ to evaluate the likelihood at.

```
theta0 <- 1
```

We can use the built in Poisson density function to obtain the likelihood for any observation, e.g.

```
dpois(y[1], 50*exp(-theta0*t[1]))
```

We obtain the likelihood for the complete data by taking the product...

```
prod(dpois(y, 50*exp(-theta0*t)))
```

Maximizing the Likelihood

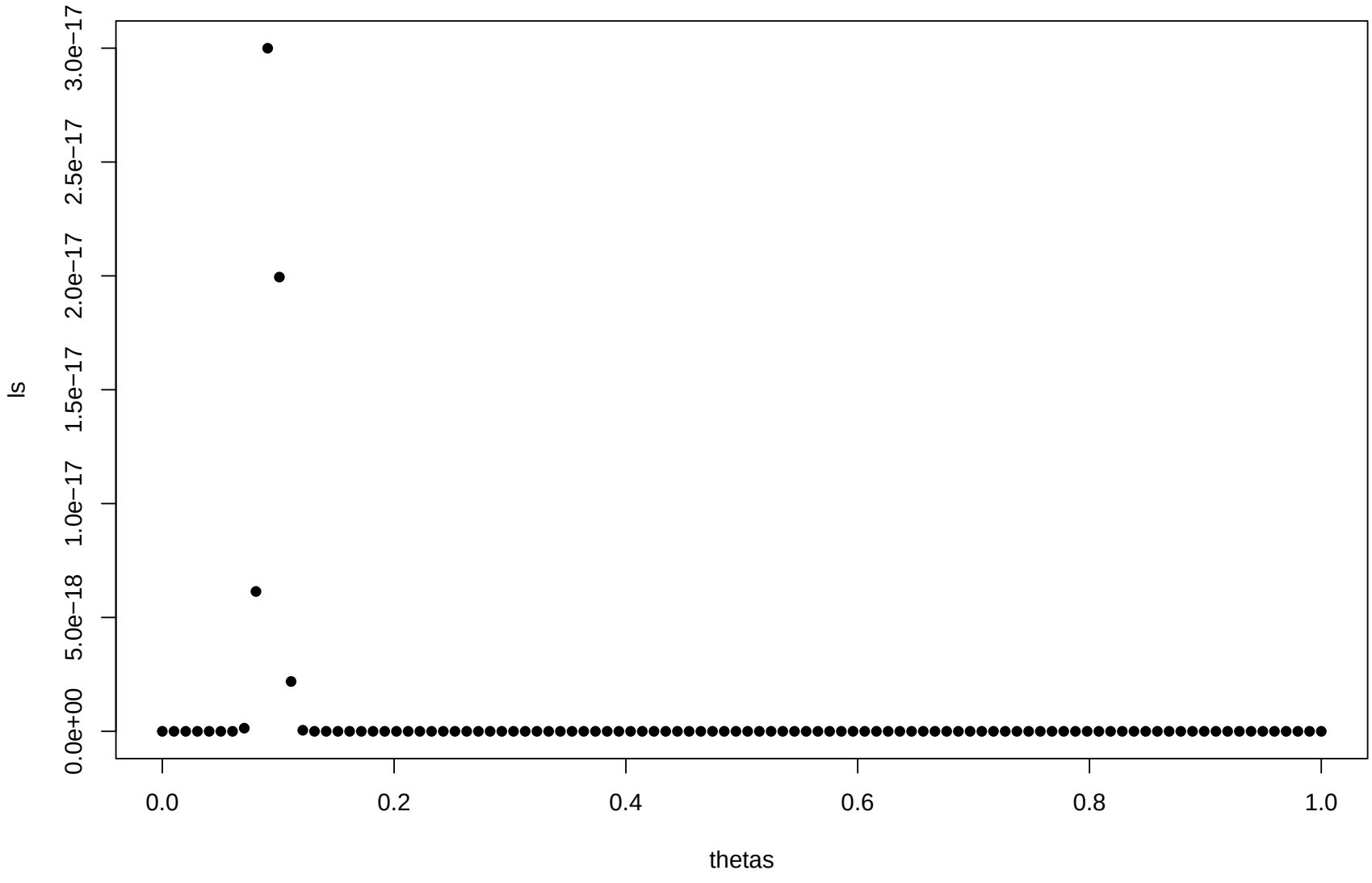
Intuitively, we find the maximum by evaluating the likelihood at a set of candidate values for θ and identifying which value achieves the maximum likelihood.

For instance, we could evaluate the likelihood for values of θ between 0 and 1.

```
thetas <- seq(0, 1, length.out = 100)
ls <- rep(NA, length(thetas))
for (i in 1:length(thetas)) {
  ls[i] <- prod(dpois(y, 50*exp(-thetas[i]*t)))
}
```

Maximizing the Likelihood

```
plot(thetas, ls, pch = 16)
```



Working on the Log Scale

We can see that something a little weird is happening with our likelihood...

```
tail(ls)
```

```
[1] 0 0 0 0 0 0
```

The last several values are exactly 0, because they correspond to products of many tiny (but not nonzero) numbers.

This motivates working on the log scale for maximum likelihood.

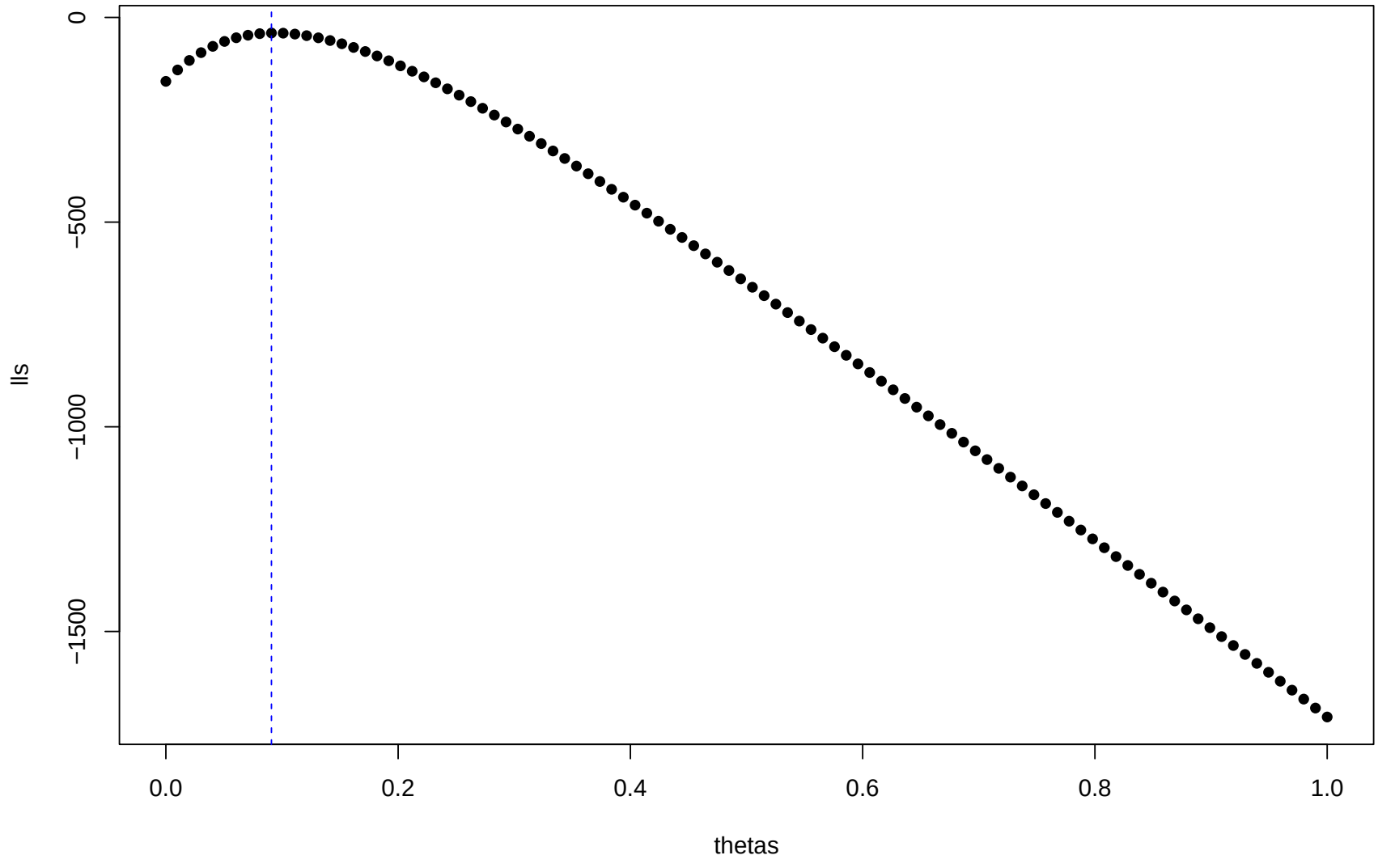
Finding the value that maximizes the log of a function $=$ finding the value that maximizes a function

Maximizing the Log Likelihood

```
lls <- rep(NA, length(thetas))
for (i in 1:length(thetas)) {
  lls[i] <- sum(dpois(y, 50*exp(-thetas[i]*t), log = TRUE))
}
```

Maximizing the Log-Likelihood

```
plot(thetas, lls, pch = 16)
abline(v = thetas[which.max(lls)], col = "blue", lty = 2)
```



This is very inefficient!

The approach we just took does give us a sense of:

- Where the maximizing value is
- The shape of the likelihood

However, it:

- Gives an imprecise solution
- Requires specification of an interval to check

Using `optim`

In practice, we will use optimization algorithms to maximize likelihoods.

In `R`, the most basic way to do this uses `optim`.

The `optim` function minimizes by default and requires:

- Starting values for the parameters that are being minimize over
- A function to minimize

Creating a Log-Likelihood Function

```
ll <- function(theta, y, t) {
  -sum(dpois(y, 50*exp(-theta*t), log = TRUE))
}
```

Note: `optim` likes to have the parameter that will be minimized over specified first.

Applying optim and Interpreting Output

```
opt <- optim(0.2, ll, y = y, t = t)
```

```
Warning in optim(0.2, ll, y = y, t = t): one-dimensional op
Mead is unreliable:
use "Brent" or optimize() directly
```

We do get a warning - but this is one that it can be ok to ignore and we will talk about why it's happening and ways around it.

Interpreting optim Output

```
opt
```

```
$par
[1] 0.09376953

$value
[1] 37.96734

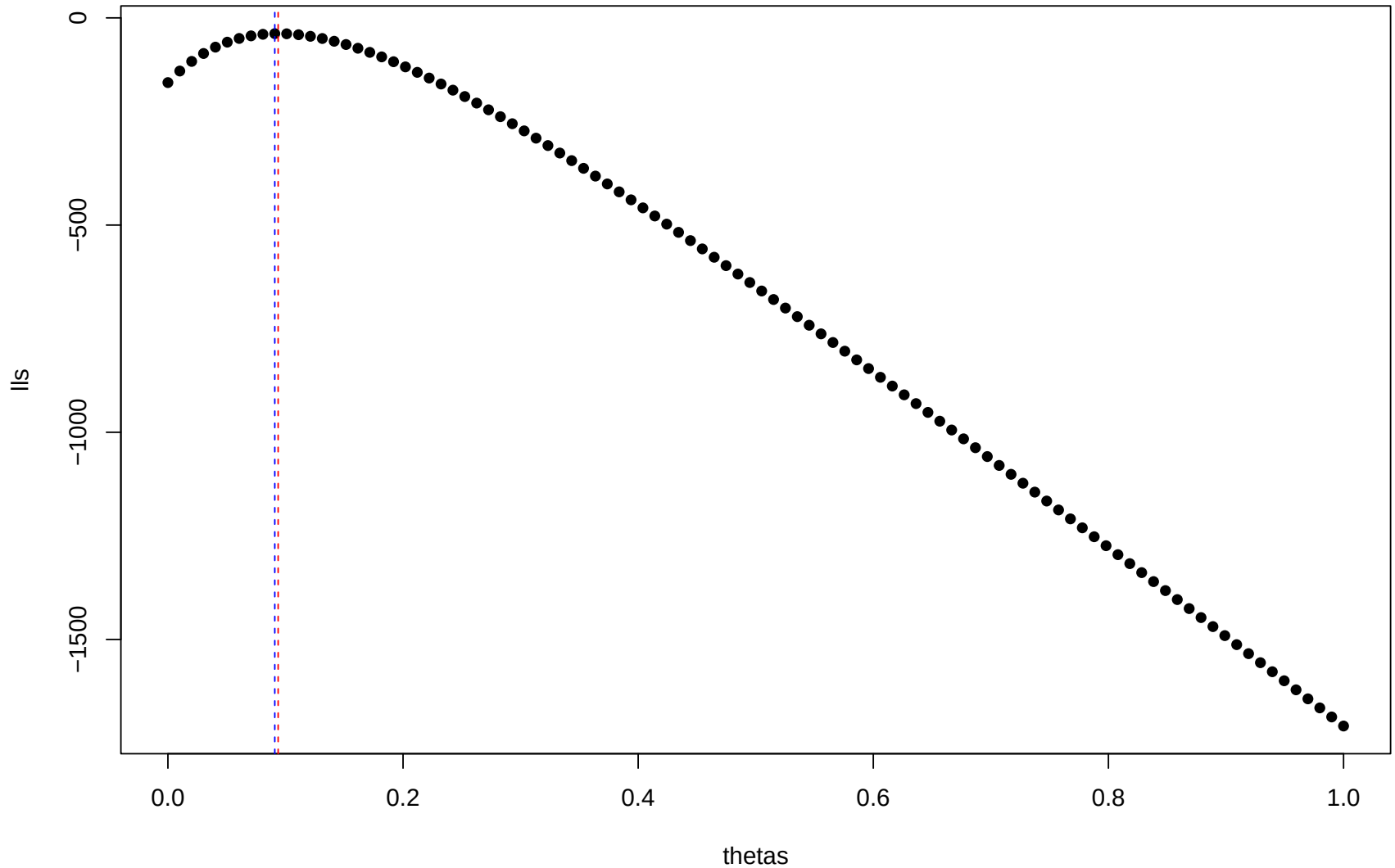
$counts
function gradient
      32       NA

$convergence
[1] 0

$message
NULL
```

Our New Estimator

```
plot(thetas, lls, pch = 16)
abline(v = thetas[which.max(lls)], col = "blue", lty = 2)
abline(v = opt$par, col = "red", lty = 2)
```



Getting Rid of the optim Error

The optim error arises when your unknown parameter is a **scalar**.

There are multiple ways to get rid of it but none use method="Nelder-Mead", with or without providing an interval to maximize over.

```
opt <- optim(0.2, ll, y = y, t = t, method = "BFGS")
opt <- optim(0.2, ll, y = y, t = t, method = "L-BFGS-B")
opt <- optim(0.2, ll, y = y, t = t, method = "CG")
```

```
opt <- optim(0.2, ll, y = y, t = t, method = "Brent",
             lower = 0, upper = 1)
opt <- optimize(ll, y = y, t = t, lower = 0, upper = 1)
```

Check how they compare!

What the heck are these methods?

Nelder-Mead, BFGS, L-BFGS-B, CG, and Brent all describe different algorithms for minimizing or maximizing a function, and which works best depends on the type of problem.

In general, L-BFGS-B is most stable, especially when we have a parameter vector.

What's an additional way we can check our solution?

We may want to verify that the gradient is in fact sufficiently close to 0 at the maximum likelihood estimate of θ.

What is the gradient? We need to write out the log likelihood and take a derivative with respect to θ.

Rewriting the Log-Likelihood

First, let's write out the log likelihood but in R so we can check our work.

```
llnew <- function(theta, y, t) {
  -sum(y*log(50) -y*theta*t +
         - 50*exp(-theta*t) - log(factorial(y)))
}
```

```
ll(theta0, y, t)
```

```
[1] 1708.895
```

```
llnew(theta0, y, t)
```

```
[1] 1708.895
```

Defining a Gradient Function

```
llgrad <- function(theta, y, t) {
  -sum(-y*t + t*50*exp(-theta*t))
}
```

Examining the gradient function at candidate solutions

```
llgrad(0.09377259, y = y, t = t)
```

```
[1] 0.1493298
```

```
llgrad(thetas[which.max(lls)], y = y, t = t)
```

```
[1] -54.96689
```

Modifying the Example to Have a Vector Parameter

Suppose we did not know the cell count before antibiotics were administered.

Then we would consider

$$y_i \overset{indep}{\sim} \text{Poisson}(\theta_2 \times \exp\{-\theta_1 t_i\})$$

Doing It Ourselves with Newton's Method

It can be helpful to work through an optimization algorithm ourselves.